

Assignment 2: AST Generator for MiniRust

Version: v1.0

1. OBJECTIVES

- Learn how to build an Abstract Syntax Tree (AST) from an ANTLR4 parse tree using the Visitor pattern.
- Understand how to map concrete syntax structures to abstract syntax nodes.
- Handle type annotations, complex expressions (such as right-associative assignments, logical operators, and unary operations), structures, loops, and control statements.
- Build a Python testing harness to run the AST generator tests.

2. REQUIRED TASKS TO COMPLETE

- **Task 1: Study the AST Node definitions**
 - Read `src/utils/AST.py` to familiarize yourself with the classes representing the AST nodes.
- **Task 2: Implement the ASTGenerator.py file**
 - Complete the class `ASTGenerator` in `src/astgen/ASTGenerator.py` (inheriting from `MiniRustVisitor`).
 - Implement visitor methods for each parser rule to traverse the parse tree and return the corresponding AST representation.
- **Task 3: Write 100 AST generator tests**
 - Create 100 test cases in `test/test-ast.py` (`test_1` to `test_100`) using the provided `check_ast` helper.
 - Test precedence, associativity, literal string escaping, control flow statements, variable and function declarations, array operations, and structure definitions/initializations.

3. AST NODE MAPPING SPECIFICATION

Your visitor must return AST instances matching the structure defined in `src/utils/AST.py` :

1. Declarations:

- `Program(decls: list[Decl])` : root of the AST containing a list of structure declarations followed by function declarations.
- `StructDecl(name: Id, fields: list[VarDecl])` : represents `struct ID { field1, field2, ... }`. Note that structure fields are modeled as `VarDecl` with `init=None` and `is_mut=False`.
- `FuncDecl(name: Id, params: list[VarDecl], return_type: Type, body: Block)` : function definition.

2. Types:

- `IntType`, `FloatType`, `BoolType`, `StringType`, `VoidType`.
- `ArrayType(element_type: Type, size: int)`.
- `StructType(name: Id)`.

3. Statements:

- `Block(stmts: list[Stmt])` : a block of statements enclosed in curly braces.

- `VarDecl(name: Id, var_type: Type, init: Expr | None, is_mut: bool) : local variable declaration.`
- `If(cond: Expr, then_stmt: Block, else_stmt: Block | If | None) : if/else branch structure.`
- `While(cond: Expr, body: Block) : while loop.`
- `For(var_name: Id, start: Expr, end: Expr, body: Block) : range-based for loop.`
- `Switch(expr: Expr, cases: list[Case], default_stmt: Block) : switch statement.`
- `Case(value: Literal, body: Block) : single case branch in a switch statement.`
- `Break() , Continue() .`
- `Return(expr: Expr | None) .`
- `ExprStmt(expr: Expr) : statement wrapping an expression.`

4. Expressions:

- `Assign(lhs: Expr, rhs: Expr) : right-associative assignment expression (returns void value in MiniRust).`
- `BinaryOp(op: str, left: Expr, right: Expr) : binary operations (arithmetic, relational, logical).`
- `UnaryOp(op: str, body: Expr) : unary operations ( - and ! ).`
- `CallExpr(name: Id, args: list[Expr]) : function call expression.`
- `ArrayCell(arr: Expr, idx: Expr) : indexing into an array (e.g. arr[idx] ).`
- `FieldAccess(obj: Expr, field: Id) : accessing a structure field (e.g. obj.field ).`
- `StructInit(name: Id, initializers: list[FieldInit]) : struct instantiation (e.g. Point { x: 1, y: 2 } ).`
- `FieldInit(name: Id, expr: Expr) : single field initializer in a struct instantiation.`
- `Id(name: str) : variable/identifier reference.`

5. Literals:

- `IntLiteral(value: int) .`
- `FloatLiteral(value: float) .`
- `BoolLiteral(value: bool) .`
- `StringLiteral(value: str) : string literal containing the evaluated text (with backslash escape sequences like \n or \t decoded into their respective characters).`
- `ArrayLiteral(elements: list[Expr]) .`

4. REQUISITES & SETUP

- Compile the grammar for Python target: `python src/run.py gen`
- Execute your unit tests: `python src/run.py test-ast`

5. PROJECT STRUCTURE

The repository layout for Assignment 2 must be structured as follows:

```
root/
├── src/
│   ├── grammar/
│   │   ├── MiniRust.g4      # Student's ANTLR4 grammar file
│   │   └── (generated files) # Generated by ANTLR (Lexer, Parser, etc.)
│   └── astgen/
```

```

├── ASTGenerator.py # Student's AST generator implementation
├── utils/
│   ├── AST.py      # AST node definitions (provided)
│   └── lexerror.py  # Lexer error utilities (provided)
├── test/
│   └── test-ast.py  # Student's 100 AST unit tests
└── requirements.txt

```

6. EVALUATION CRITERIA & GRADING

- **AST Generation Correctness (100%):** Grade is based on passing the automated test cases run against your submitted grammar and `ASTGenerator.py` implementation.
- **Test Cases Penalty:** Writing at least 100 unit tests in `test/test-ast.py` is mandatory. A penalty of 10% of the total assignment grade will be deducted if `test/test-ast.py` is missing or contains fewer than 100 test cases.

Test Distribution Table

The 100 tests in `test/test-ast.py` are distributed by language feature as follows to guide your implementation:

Structure	Test Cases	Description
Declarations & Definitions	test_1 - test_9	Root program layout, function declarations (void/typed), struct definitions, basic variables (let/let mut), local type inference, and mutable uninitialized variables.
Control Flow Structures	test_10 - test_22	Conditional branching (if-else lồng sâu), loop statements (while, for-in), control actions (break, continue, return), and nested lexical blocks { }.
Switch Statement (C-style)	test_23 - test_27	C-style switch branching including logic expression checks, case fall-through chains (case 1: case 2: ...), and nested blocks in default clauses.
Arrays & Struct Instantiation	test_28 - test_37	Single/multidimensional arrays ([[i32; 2]; 3]), struct init syntax (Point { x: 1, y: Point { ... } }), and field access/indexing boundaries.
Literals & Escape Sequences	test_38 - test_47	Booleans, floating-point scientific exponential notations (e), and advanced string literals containing nested escape sequences (\n, \t, \", \\\).
Function Calls & Return	test_48 - test_52	Library/user function invocations (with or without arguments) and returns carrying complex arithmetic results or call expressions.
Operators & Precedence	test_53 - test_81	Full precedence ladder: Unary (-, !), Multiplicative (*, /, %), Additive (+, -), Relational and Non-associative Equality (==, !=, <, <=, >, >=), Logical AND/OR, and Right-associative Assignment.
Nested Complex Accesses	test_82 - test_96	Deeply nested dereferences combining arrays, member fields, and function calls (e.g. arr[matrix[0][0]], func(a)[0].b).

Chained Assignment Expressions	test_97 - test_100	Right-associative assignment chains ($x = y = 5$), assignments inside conditions, and member assignment targets ($a.b = c.d = 1$).
---------------------------------------	-----------------------	--

7. DELIVERABLES

Students must submit:

- File `src/grammar/MiniRust.g4` containing the ANTLR4 grammar definition.
- File `src/astgen/ASTGenerator.py` containing the AST generation visitor.
- File `test/test-ast.py` containing exactly 100 AST test cases.

8. DEADLINE

- **Submission Deadline:** Refer to the [deadlines.json](#) Gist file for the exact due date for Assignment 2.

9. CHANGELOG

- First release under v1.0.